

LAPORAN PRAKTIKUM 2
PEMROGRAMAN BERORIENTASI OBJEK
RELASI ANTAR OBJEK (CLASS DIAGRAM & ASOSIASI)



Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

NURFIAH, S.ST, M.KOM

Asisten Praktikum :

EZZA ADDINI

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadiran Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena taufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan Pemrograman Berorientasi Objek (PBO) menggunakan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Relasi Antar Objek (Class Diagram & Asosiasi) serta penerapannya dalam pembuatan program sederhana berupa sistem perbankan mini. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar Pemrograman Berorientasi Objek dan bahasa pemrograman Java.

Padang, September 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB 1 PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN	
2.1 Langkah Kerja Praktikum.....	3
2.1.1 Membuat Package dan Class.....	3
2.1.2 Membuat Class Rekening.....	3
2.1.3 Membuat Class Transaksi.....	6
2.1.4 Membuat Class Main.....	7
2.2 Tugas Praktikum.....	12
2.2.1 Tugas 1.....	12
2.2.2 Tugas 2.....	13
2.2.3 Output Tugas 1 dan Tugas 2.....	14
BAB 3 KESIMPULAN	
3.1 Kesimpulan.....	15
3.2 Link Repository.....	15

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, setiap objek dapat memiliki hubungan dengan objek lainnya untuk membentuk sebuah sistem yang lebih terstruktur. Salah satu konsep yang digunakan adalah relasi asosiasi, yaitu hubungan antara satu objek dengan objek lainnya. Pada Praktikum 2 ini, konsep tersebut diterapkan pada sistem perbankan dengan menghubungkan class Rekening dan Transaksi, di mana satu rekening dapat memiliki banyak transaksi.

Pada praktikum ini juga dilakukan penerjemahan Class Diagram ke dalam kode program Java serta penggunaan ArrayList untuk mengelola kumpulan objek secara dinamis. Setiap transaksi setor dan tarik tunai yang berhasil akan dibuat sebagai objek Transaksi dan disimpan sebagai riwayat pada objek Rekening. Dengan adanya fitur cetakMutasi(), riwayat transaksi tersebut dapat ditampilkan kembali sehingga penerapan relasi antar objek dapat dilihat secara langsung dalam program.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Membaca dan menerjemahkan Class Diagram ke dalam kode program Java.
2. Memahami dan mengimplementasikan relasi Asosiasi (one to many)
3. Memanajemen koleksi objek di dalam objek lain menggunakan tipe data bentukan dan struktur data dinamis (ArrayList).

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami hubungan antar objek dalam Java melalui penerapan relasi asosiasi *one to many* antara Rekening dan Transaksi.
2. Meningkatkan pemahaman dalam menerjemahkan Class Diagram menjadi struktur class, atribut, constructor, dan method pada program Java.
3. Memahami penggunaan ArrayList untuk menyimpan dan mengelola banyak objek Transaksi sebagai riwayat transaksi dalam sebuah rekening.

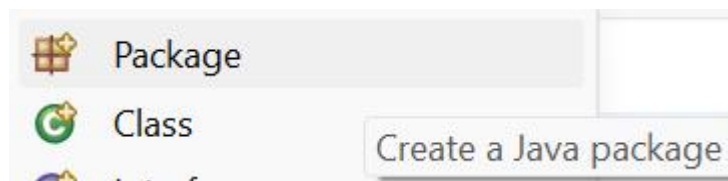
BAB II

PEMBAHASAN

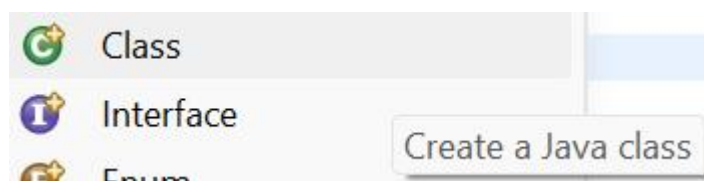
2.1 Langkah Kerja Praktikum

2.1.1 Membuat Package dan Class

1. Buat Package dengan nama Praktikum2

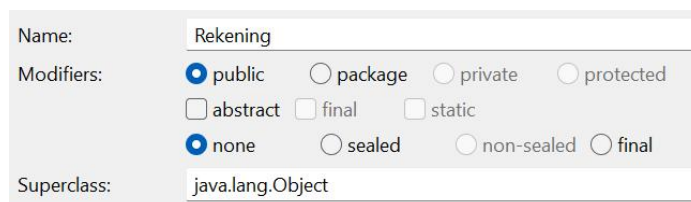


2. Setelah membuat Package, lanjutkan dengan membuat Class.



2.1.2 Membuat Class Rekening

1. Buat class baru dengan nama Rekening pada package Praktikum2. Pada class ini nantinya akan dibuat data dan method yang berhubungan dengan sebuah rekening.



2. Scanner dan ArrayList digunakan untuk kebutuhan input dan penyimpanan data rekening pada program.

```
package Praktikum1;  
import java.util.Scanner;  
import java.util.ArrayList;
```

2. Setelah itu, tambahkan tiga atribut. Atribut nomorRekening dan namaPemilik menggunakan tipe data String karena data yang disimpan berupa teks, sedangkan saldo menggunakan tipe data double karena saldo berupa angka dan bisa memiliki nilai desimal.

```
public class Rekening {  
    String nomorRekening;  
    String namaPemilik;  
    double saldo;
```

Ketiga atribut nantinya digunakan untuk menyimpan nomor rekening, nama pemilik, dan jumlah saldo yang dimiliki oleh setiap objek rekening.

3. `// Implementasi Asosiasi (1-to-many)`
`ArrayList<Transaksi> riwayatTransaksi;`

Digunakan untuk menyimpan seluruh riwayat transaksi pada satu rekening. Konsep ini menerapkan asosiasi 1-to-many, yaitu satu objek Rekening dapat memiliki banyak objek Transaksi.

4. Selanjutnya buat constructor dengan nama yang sama seperti nama class. Constructor ini menerima tiga parameter, yaitu nomor, nama, dan saldoAwal. Nilai dari parameter tersebut kemudian dimasukkan ke atribut rekening masing-masing.

```
public Rekening(String nomor, String nama, double saldoAwal) {  
    this.nomorRekening = nomor;  
    this.namaPemilik = nama;  
    this.saldo = saldoAwal;
```

Ketiga baris diatas digunakan untuk menginisialisasi data awal rekening saat objek Rekening dibuat.

5. Buat method setorTunai()

```
public void setorTunai(double nominal) {  
    if (nominal > 0) {  
        saldo += nominal;  
        String idTrx = "TRX-S-" + System.currentTimeMillis();  
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit",  
            nominal);  
        riwayatTransaksi.add(trxBaru);
```

```

        System.out.println("Setor tunai Rp" + nominal + "
        berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih
        dari 0!");
    }
}

```

Method `setorTunai()` digunakan untuk melakukan penyetoran uang ke rekening. Program mengecek apakah nominal setor lebih dari 0, kemudian menambahkan nominal tersebut ke saldo. Jika berhasil, transaksi dicatat sebagai Kredit dan disimpan ke dalam `riwayatTransaksi`.

6. Buat method `tarikTunai()`

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal
        nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak
        mencukupi. Saldo Anda: Rp" + saldo);
    } else { saldo -= nominal;
        System.out.println("Tarik tunai berhasil.");
        System.out.println("Jumlah penarikan: Rp" + nominal);
        System.out.println("Saldo sekarang: Rp" + saldo);

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit",
        nominal);
        riwayatTransaksi.add(trxBaru);
    }
}

```

Method `tarikTunai()` digunakan untuk melakukan penarikan uang dari rekening. Program mengecek apakah nominal penarikan sudah memenuhi batas minimal dan saldo mencukupi. Jika penarikan berhasil, saldo akan dikurangi dan transaksi dicatat sebagai Debit ke dalam `riwayatTransaksi`.

7. Buat method `cekInformasi()`

```

public void cekInformasi() {
    System.out.println("=== INFO REKENING ===");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("=====");
}

```

```
}
```

Method `cekInformasi()` digunakan untuk menampilkan informasi rekening yang sedang aktif. Informasi yang ditampilkan meliputi nomor rekening, nama pemilik, dan saldo akhir rekening.

8. Buat method `cetakMutasi()`

```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening  
        ini");  
    } else {  
        for (Transaksi transaksi : riwayatTransaksi) {  
            transaksi.cetakDetail();  
        }  
    }  
}
```

Method `cetakMutasi()` digunakan untuk menampilkan riwayat transaksi pada rekening. Program mengecek apakah terdapat transaksi, kemudian menggunakan perulangan `for-each` untuk menampilkan detail setiap transaksi melalui method `cetakDetail()`. Jika belum ada transaksi, program akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

2.1.3 Membuat Class Transaksi

1. Buat class baru dengan nama `Transaksi` pada package `Praktikum2`.

Name:	Transaksi			
Modifiers:	☒ public	☐ package	☐ private	☐ protected
	☐ abstract	☐ final	☐ static	
	☒ none	☐ sealed	☐ non-sealed	☐ final

2. Tambahkan tiga atribut yaitu `idTransaksi` untuk menyimpan identitas atau ID transaksi, `jenis` untuk menyimpan jenis transaksi seperti Debit atau Kredit, dan `nominal` untuk menyimpan jumlah uang yang terlibat dalam transaksi.

```
public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;
```

3. Selanjutnya, buat constructor yang digunakan untuk menginisialisasi data transaksi saat objek Transaksi dibuat. Setiap objek transaksi memiliki ID, jenis transaksi, dan jumlah transaksi yang sesuai.

```
// Constructor
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

4. Setelah itu, buat method cekDetail() yang digunakan untuk menampilkan detail transaksi ke layar.

```
public void cetakDetail() {
    System.out.println("ID: " + idTransaksi + " | Jenis: " +
        jenis + " | Nominal: Rp" + nominal);
}
```

2.1.4 Membuat Class Main

1. Membuat scanner dan variabel program

```
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;
```

Kode diatas digunakan untuk menyiapkan input pengguna dengan Scanner, menyimpan daftar rekening menggunakan ArrayList, menentukan akun yang sedang aktif melalui akunAktif, dan mengatur program agar tetap berjalan dengan isRunning.

2. Membuat menu utama. Perulangan while digunakan agar menu terus ditampilkan selama program masih berjalan.

```

System.out.println("== SISTEM PERBANKAN MINI ==");

while (isRunning) {
    System.out.println("\n=== Menu Utama ===");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Cetak Mutasi (Riwayat)");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");

```

3. Input dari pengguna disimpan ke dalam variabel pilihan. Selanjutnya, switch digunakan untuk menentukan proses yang dijalankan sesuai dengan nomor menu yang dipilih.

```

int pilihan = input.nextInt();
    input.nextLine(); // Membersihkan buffer enter

    switch (pilihan) {

```

4. Membuat proses buka rekening

```

case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();

    Rekening rekeningBaru = new Rekening(no, nama, saldo);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    System.out.println("Rekening berhasil ditambahkan.");
    break;

```

Pada menu ini, pengguna memasukkan nomor rekening, nama pemilik, dan saldo awal. Data tersebut digunakan untuk membuat objek Rekening, kemudian disimpan ke daftarRekening dan dijadikan sebagai akunAktif.

5. Membuat proses setor tunai

```

case 2:

    if (akunAktif == null) {

```

```

        System.out.println("Error: Mohon maaf, Anda belum memiliki
        nomor rekening!");
    } else {
        System.out.print("Masukkan nominal setor: ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;

```

Sebelum melakukan setor tunai, program mengecek apakah sudah ada rekening yang aktif. Jika belum ada, program menampilkan pesan kesalahan. Jika sudah ada, nominal setor dimasukkan dan method `setorTunai()` dipanggil.

6. Membuat proses tarik tunai

```

case 3:

if (akunAktif == null) {
    System.out.println("Error: Anda belum membuka rekening!");
} else {
    System.out.print("Masukkan nominal tarik: ");
    double tarik = input.nextDouble();
    akunAktif.tarikTunai(tarik);
}
break;

```

Program mengecek apakah sudah ada akun aktif, lalu meminta nominal penarikan dan menjalankan method `tarikTunai()`.

7. Membuat proses cek informasi

```

case 4:

if (akunAktif == null) {
    System.out.println("Error: Anda belum membuka rekening!");
} else {
    akunAktif.cekInformasi();
}
break;

```

Jika ada akun aktif, program menjalankan `cekInformasi()` untuk menampilkan data rekening.

8. Membuat proses mengganti akun aktif

```

case 5 :
    if (daftarRekening.isEmpty()) {
        System.out.println("Belum ada rekening yang tersedia.");
    } else {
        System.out.print("Masukkan nomor rekening yang ingin digunakan: ");
        String nomorCari = input.nextLine();
        boolean ditemukan = false;
        for (Rekening rekening : daftarRekening) {
            if (rekening.nomorRekening.equals(nomorCari)) {
                akunAktif = rekening;
                ditemukan = true;
                System.out.println("Berhasil mengganti akun.");
                System.out.println("Akun aktif: " + akunAktif.namaPemilik);
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Rekening tidak ditemukan.");
        }
    }
    break;

```

Pengguna memasukkan nomor rekening, kemudian program mencari rekening tersebut di daftarRekening dan menjadikannya sebagai akun aktif.

9. Membuat proses cetak mutasi

```

case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;

```

Case 6 digunakan untuk menampilkan riwayat transaksi atau mutasi rekening dengan menjalankan method cetakMutasi() pada akun yang sedang aktif

10. Membuat proses menghentikan program

```

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;

default:
    System.out.println("Pilihan tidak valid!");

```

Default digunakan untuk menampilkan pesan kesalahan jika pengguna memasukkan pilihan menu yang tidak tersedia.

11. Program diakhiri dengan menutup scanner

```
input.close();
```

2.2 Tugas Praktikum

2.2.1 Tugas 1 (Integrasi Rekam Jejak Penarikan)

Menambahkan logika perekaman objek Transaksi pada method tarikTunai(double nominal) di dalam class Rekening. Setiap penarikan yang berhasil dicatat sebagai jenis transaksi "Debit" dengan kode yang menggunakan prefiks "TRX-T-", kemudian disimpan ke dalam riwayatTransaksi.

➤ Kode Program :

```
String idTrx = "TRX-T-" + System.currentTimeMillis();
Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
riwayatTransaksi.add(trxBaru);
```

➤ Langkah Kerja :

1. Program mengecek terlebih dahulu apakah nominal penarikan kurang dari Rp10.000 atau lebih besar dari saldo. Jika penarikan berhasil, saldo dikurangi sesuai nominal yang ditarik.

2. Setelah penarikan berhasil, tambahkan kode untuk membuat ID transaksi dengan awalan TRX-T-.

```
String idTrx = "TRX-T-" + System.currentTimeMillis();
```

3. Buat objek Transaksi dengan jenis "Debit" dan nominal penarikan.

```
Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
```

4. Tambahkan objek tersebut ke riwayatTransaksi agar transaksi penarikan tersimpan.

```
riwayatTransaksi.add(trxBaru);
```

2.2.2 Tugas 2 (Fitur Cetak Mutasi Rekening)

Membuat method cetakMutasi() pada class Rekening untuk menampilkan seluruh riwayat transaksi. Method menggunakan perulangan for-each untuk menelusuri riwayatTransaksi dan memanggil cetakDetail() pada setiap objek Transaksi. Jika riwayat transaksi kosong, program menampilkan pesan "Belum ada transaksi pada rekening ini".

➤ Kode Program :

```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening  
        ini");  
    } else {  
        for (Transaksi transaksi : riwayatTransaksi) {  
            transaksi.cetakDetail();  
        }  
    }  
}
```

➤ Langkah Kerja :

1. Buka file Rekening.java, kemudian buat method baru bernama cetakMutasi().

2. Cek apakah riwayatTransaksi masih kosong menggunakan isEmpty().

```
if (riwayatTransaksi.isEmpty()) {
```

3. Jika belum ada transaksi, tampilkan pesan "Belum ada transaksi pada rekening ini".

```
System.out.println("Belum ada transaksi pada rekening ini");
```

4. Jika ada transaksi, gunakan perulangan for-each untuk mengambil setiap data transaksi.

```
} else {  
    for (Transaksi transaksi : riwayatTransaksi) {
```

5. Panggil method cetakDetail() untuk menampilkan detail dari setiap transaksi.

```
transaksi.cetakDetail();
```

6. Tambahkan pilihan Cetak Mutasi (Riwayat) pada menu di Main.java, kemudian panggil cetakMutasi() saat menu tersebut dipilih.

```
System.out.println("6. Cetak Mutasi (Riwayat)");
```

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

2.2.3 Output Tugas 1 dan Tugas 2

```
=== Menu Utama ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan nominal tarik: 100000
Tarik tunai berhasil.
Jumlah penarikan: Rp100000.0
Saldo sekarang: Rp778953.0

=== Menu Utama ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
ID: TRX-S-1789708314047 | Jenis: Kredit | Nominal: Rp89877.0
ID: TRX-T-1789708320472 | Jenis: Debit | Nominal: Rp100000.0
```

Output tersebut menunjukkan bahwa penarikan tunai sebesar Rp100.000 berhasil, sehingga saldo berkurang menjadi Rp778.953. Saat memilih menu 6. Cetak Mutasi (Riwayat), program menampilkan dua transaksi yang tersimpan, yaitu transaksi Kredit sebesar Rp89.877 dan transaksi Debit sebesar Rp100.000.

BAB III

KESIMPULAN

3.1 Kesimpulan

Pada Praktikum 2 ini, dapat dipahami bahwa relasi antar objek dapat digunakan untuk menghubungkan satu objek dengan objek lainnya. Pada program yang dibuat, satu objek Rekening dapat memiliki banyak objek Transaksi yang disimpan dalam ArrayList sebagai riwayat transaksi. Selain itu, transaksi setor dan tarik tunai dapat dicatat dengan jenis Kredit untuk setor dan Debit untuk tarik.

Praktikum ini juga memberikan pemahaman tentang cara membuat dan mengelola riwayat transaksi dalam program Java. Dengan adanya fitur cetakMutasi(), setiap transaksi yang telah dilakukan dapat ditampilkan kembali sehingga data transaksi pada rekening dapat diketahui dengan lebih mudah.

3.2 Link Repository

https://github.com/kinayanovryamanda7-blip/PrakPBO_B_2511531016